AI Resume Analyzer and Job Recommendation System

Major Project 1 · Project Report

Student Name	Nand P Bhojani
Project	AI Resume Analyzer and Job Recommendation System
Submission ID	12569237
Submission Date	25 October 2026
Live Application	https://nand-resume-analyzer.streamlit.app/
GitHub Repository	https://github.com/Mr-360-17/ai-resume-analyzer

1. Project Objective

The goal is an NLP-based web application that helps students understand how well their resume matches a chosen job role. The user uploads a PDF or DOCX resume and the system returns:

- a match score for every job role, and the top three recommended roles;
- the skills found in the resume, grouped by category;
- the skills that are missing, or weak, for a selected target role;
- a week-by-week learning roadmap and a downloadable PDF report.

2. Problem Statement

Students often do not know whether their resume shows the skills a role expects, and recruiters need a quick way to compare job-related information across many resumes. This project builds an educational matching system that looks only at job-related content: skills, projects, education and relevant experience. It is a guidance tool, not a hiring decision maker.

3. Tools and Technologies

Project part	Tool used
Programming language	Python 3.11
PDF and DOCX text extraction	pypdf, python-docx
Text cleaning and skill matching	Python regular expressions with a controlled skill dictionary
Text vectors and similarity	scikit-learn (`TfidfVectorizer`, `cosine_similarity`)
Data handling	Pandas, NumPy
Charts	Plotly
User interface	Streamlit
PDF report generation	ReportLab
Deployment and version control	Streamlit Community Cloud, Git and GitHub

4. System Architecture and Workflow

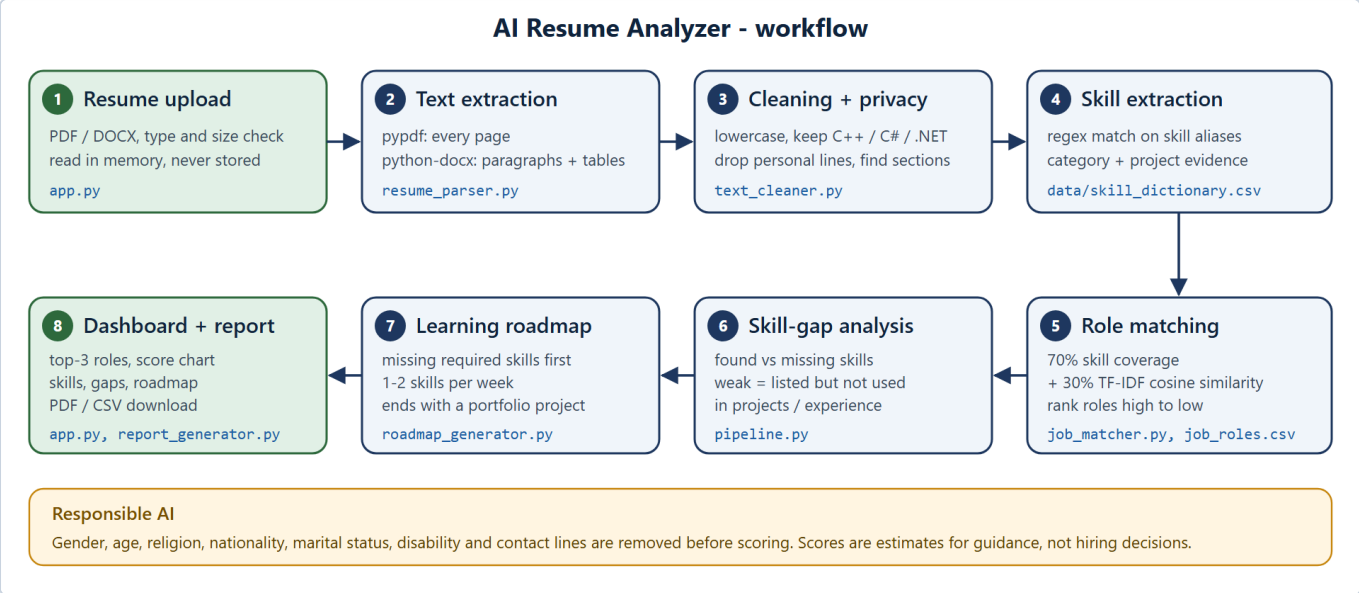


Figure 1. End-to-end workflow, from upload to dashboard and report.

5. Module Implementation

Module	File	What it does
1. Resume upload	`app.py`, `resume_parser.py`	Accepts PDF/DOCX only, rejects empty, corrupt and over-5 MB files. Files are read from memory and never written to disk.
2. Text extraction and cleaning	`resume_parser.py`, `text_cleaner.py`	Extracts every PDF page and every DOCX paragraph and table. Lowercases the text and removes stray symbols while keeping C++, C#, .NET, Node.js and CI/CD intact. Detects sections such as Education, Skills, Projects and Experience.
3. Skill extraction	`skill_extractor.py`	Matches 76 skills in 10 categories using their aliases (e.g. *sklearn* → scikit-learn). Custom word boundaries stop false hits such as "ml" inside "html". Also detects degree level and experience duration.
4. Job-role dataset	`data/job_roles.csv`	9 roles, each with required skills, nice-to-have skills and a short description.
5. Matching and recommendation	`job_matcher.py`	Scores the resume against every role and ranks them from highest to lowest. A pasted job description becomes an extra custom role.
6. Skill-gap analysis	`pipeline.py`, `roadmap_generator.py`	Lists found, missing and weak skills for the target role, then builds a learning roadmap.
7. Streamlit dashboard	`app.py`, `report_generator.py`	Upload, target-role selection, extracted skills, score chart, recommended roles, skill gap, roadmap, and PDF/CSV downloads.

6. Datasets

Skill dictionary (`data/skill_dictionary.csv`). 76 manually verified skills, each with a category, the aliases used to detect it and a learning topic used by the roadmap. Categories: Programming, Databases, Data & Analytics, Machine Learning, Deep Learning & AI, NLP, Computer Vision, Web & APIs, Cloud & DevOps, and Tools.

Job roles (`data/job_roles.csv`). Data Analyst, Data Scientist, Machine Learning Engineer, AI Engineer, NLP Engineer, Computer Vision Engineer, Python Developer, Web Developer, and Cloud & DevOps Engineer. Each role lists 6-8 required skills (most important first) and 2-4 nice-to-have skills.

Sample resumes (`data/sample_resumes/`). Five fictional resumes (3 PDF, 2 DOCX) with all personal information replaced by placeholders, used for testing and the demo.

7. Matching and Scoring Method

```
Match score = 70% × Skill coverage + 30% × Text similarity
```

- **Skill coverage** is the share of the role's skills found in the resume. Required skills count 1 and nice-to-have skills count 0.5. It is easy to explain: "you have 7 of the 8 required skills".
- **Text similarity** is the cosine similarity between TF-IDF vectors of the cleaned resume and the role profile (description, skill names and aliases). TF-IDF gives high weight to words that are frequent in one document but rare across all documents. Cosine similarity measures the angle between the two vectors, so it compares *what* is written rather than how long the text is. This part rewards related wording that the dictionary does not list.
- **Weak skills** are skills that appear in the resume's skills list but never in its Projects or Experience sections.
- **Roadmap rules.** Missing required skills come first, in the order the role lists them, followed by nice-to-have skills. Up to six skills get one week each; more than six get two per week. The plan ends with a portfolio-project week that uses the new skills.

The 70/30 weighting was chosen by hand and is a design decision, not a trained value, because no labelled resume-to-role dataset was available. A resume with every listed skill still scores roughly 75-85%, because its wording never matches the role text exactly.

8. Application Screenshots

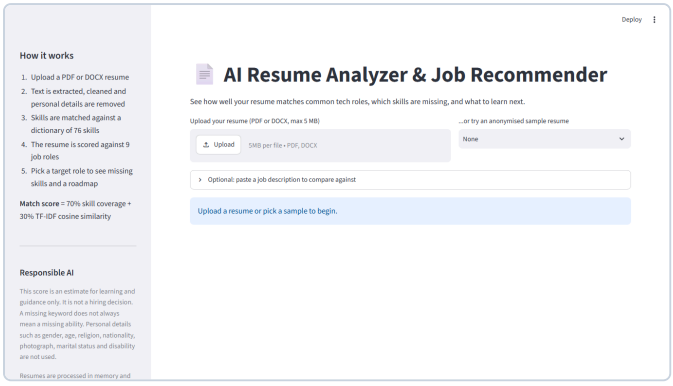


Figure 2. Home page: upload a resume or pick a sample.

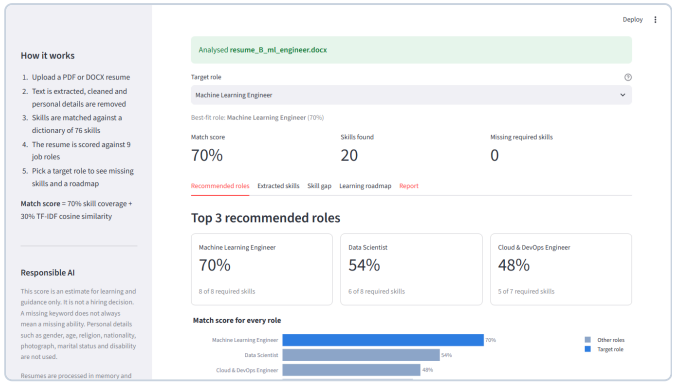


Figure 3. Match score and top three recommended roles.



Figure 4. Match score for every role, with the target role highlighted.

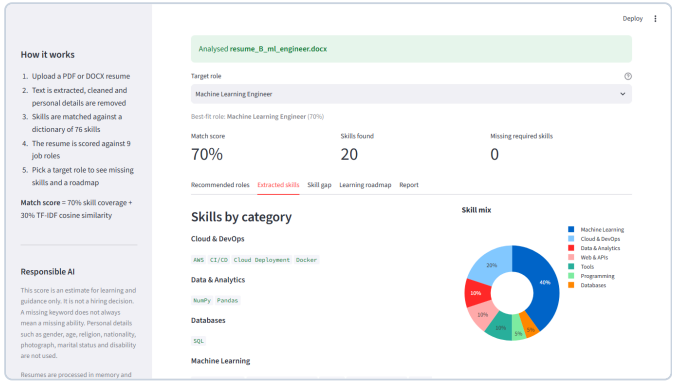


Figure 5. Extracted skills grouped by category.

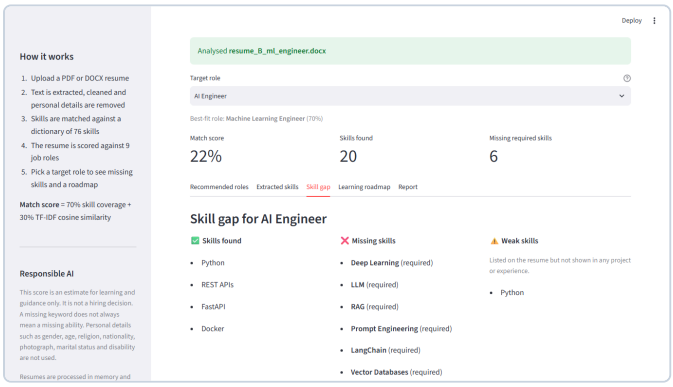


Figure 6. Skill gap for the target role AI Engineer.

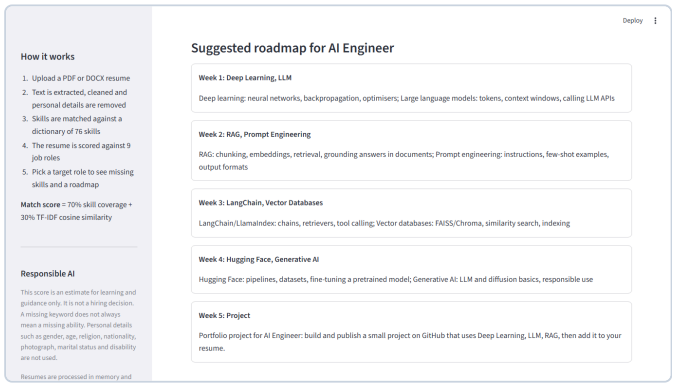


Figure 7. Week-by-week learning roadmap.

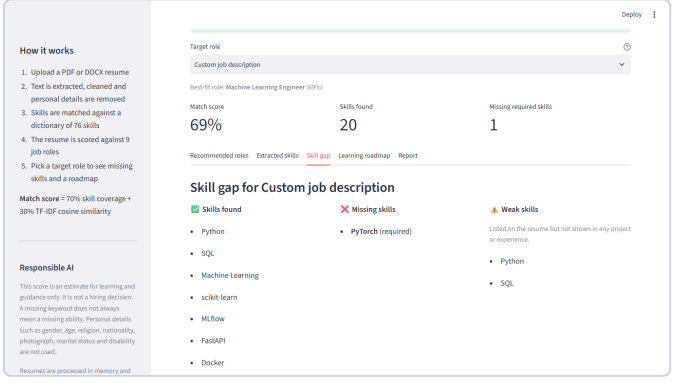


Figure 8. A pasted job description scored as its own role.

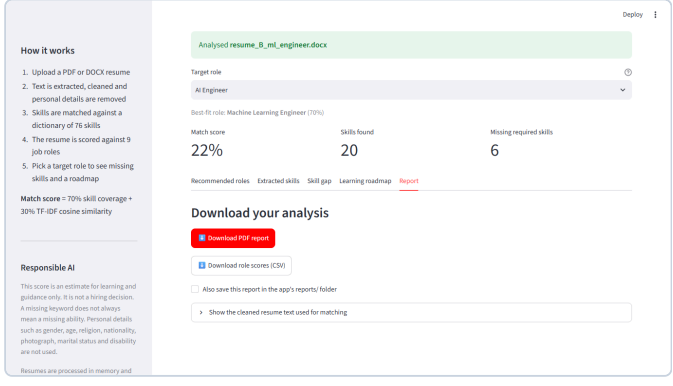


Figure 9. PDF report and CSV downloads.

9. Testing and Evaluation

The script `tests/run_tests.py` checks each point in the brief's evaluation list, and writes the expected-versus-actual sheet `tests/test_cases.csv`.

Test resume	Expected top role	Actual top role	Score	Result
Resume A (PDF)	Data Analyst	Data Analyst	78%	Pass
Resume B (DOCX)	Machine Learning Engineer	Machine Learning Engineer	70%	Pass
Resume C (PDF)	NLP Engineer	NLP Engineer	80%	Pass
Resume D (DOCX)	Computer Vision Engineer	Computer Vision Engineer	81%	Pass
Resume E (PDF)	Web Developer	Web Developer	77%	Pass

Criterion (from brief)	How it was tested	Result
Extraction quality	Text extracted from all 5 samples, PDF and DOCX	Pass
Skill precision	Detected skills that a human reader agrees are present	0.97
Skill recall	Skills a human reader listed that the system also found	1.00
Role ranking	Expected top role equals actual top role	5 / 5
Score consistency	Adding role skills raised the Data Analyst score from 36% to 64%; an unrelated skill left coverage unchanged	Pass
Fairness	Adding gender, date of birth, marital status, religion, nationality and email lines changed no score	Pass
Robustness	.txt, empty, corrupt and over-5 MB files are rejected with a clear message; C++, C#, .NET kept as skills	Pass
Usability	Plain-language labels, every score split into its two parts, and an in-app explanation of the formula	Manual check

The only extra skill detected was "Classification" on resumes C and D. It was triggered by "text classification" and "image classification", which is arguably correct.

10. Responsible AI

- The tool is for guidance only. It does not make automatic hiring or rejection decisions, and the app says so on every page.
- Lines mentioning gender, age or date of birth, religion, nationality, caste, marital status, disability or photograph are removed before scoring, along with email and phone lines.
- Only job-related skills, education, projects and experience are evaluated.
- Resumes are processed in memory and never stored. Reports are saved only if the user ticks a box.
- The app reminds users that a missing keyword does not always mean a missing ability.

11. Limitations and Future Scope

Limitation	Possible improvement
Keyword matching misses skills described in other words (e.g. "built neural nets").	Sentence Transformers for semantic matching.
The skill dictionary and roles are hand-made, not live job-market data.	Grow the dictionary from user feedback and real job postings.
Scanned, image-only PDFs have no text layer.	Add OCR.
Section detection relies on common headings.	spaCy or an LLM to classify resume sections.
The 70/30 weights were set by hand.	Learn them from labelled resume-role pairs if such data becomes available.

Other extensions: LLM-written resume feedback with a controlled prompt, a FastAPI backend with Docker, and user login with saved reports.

12. Conclusion

The project meets all minimum features in the brief. It reads PDF and DOCX resumes, extracts 76 dictionary skills, compares the resume with 9 roles, recommends the top three, explains the skill gap for a chosen role and produces a learning roadmap and a PDF report. The score is transparent (skill coverage plus TF-IDF cosine similarity), the evaluation passes every check on the sample set, and the app follows the brief's Responsible AI rules. It is deployed on Streamlit Community Cloud with the full source code on GitHub.

13. Submission Contents

Deliverable	Location
Working Streamlit application	https://nand-resume-analyzer.streamlit.app/
Complete source code and GitHub repository	`source_code/` · github.com/Mr-360-17/ai-resume-analyzer
Job-role and skill datasets	`source_code/data/`
Sample resumes (personal information removed)	`source_code/data/sample_resumes/`
requirements.txt and README	`source_code/`
Architecture diagram	`source_code/docs/architecture.png`
Testing sheet	`source_code/tests/test_cases.csv`
Project report and demonstration video	This PDF · `Nand_AI_Resume_Analyzer_Demo.mp4`